Lottery Scheduler Implementation in xv6

1. Introduction

Lottery scheduling is a probabilistic CPU scheduling algorithm that allocates processor time based on ticket ownership. Each runnable process is assigned several lottery tickets, and during each scheduling cycle, the operating system randomly selects one ticket from the total pool. The process holding the winning ticket is chosen to run.

This approach provides proportional fairness because processes with more tickets have a higher probability of receiving CPU time. Unlike round-robin scheduling, which gives each process equal turns, lottery scheduling allows flexible CPU allocation based on priority or resource needs.

The objective of this project was to modify the xv6 operating system to implement a lottery scheduler by adding ticket management and monitoring, process statistics tracking, and new system calls to control and monitor scheduling behavior.

2. Implementation

To implement lottery scheduling in xv6, several kernel components were modified.

First, the struct proc data structure in proc.h was expanded by adding two new fields:

- tickets — stores the number of tickets assigned to each process
- ticks — tracks how many times the process has been scheduled

Each new process was initialized with one default ticket in allocproc(), and child processes created through fork() inherited their parent's ticket count.

Two new system calls were added:

settickets(int number)

This system call allows a process to set its own ticket count. Invalid values return -1, while valid values update the process ticket allocation.

getpinfo(struct pstat *ps)

This system call returns scheduling statistics for all processes, including:

- Process ID
- Ticket count
- Tick count
- Active status

A new pstat structure was created to store this data.

The default round-robin scheduler in scheduler() was then replaced with lottery scheduling logic:

1. Calculate the total number of tickets among all runnable processes
2. Generate a random winning ticket
3. Iterate through runnable processes until the winning ticket range is reached
4. Schedule the selected process
5. Increment its tick counter

This implementation ensures that CPU scheduling frequency is approximately proportional to ticket allocation.

3. Testing Method

To test scheduler fairness, a custom user program called lotterytest was created.

The program:

- Forked three child processes
- Assigned ticket values of:
    - Process A = 30 tickets
    - Process B = 20 tickets
    - Process C = 10 tickets
- Kept each process in a busy loop
- Used getpinfo() to measure how often each process was scheduled

The expected ratio was 3:2:1, meaning the 30-ticket process should run most frequently, while the 10-ticket process should run least frequently.

Five separate test runs were conducted to reduce randomness and improve reliability.

Run 1:

```
moubarak@ubuntu-os: ~/xv6-public — ma...
~/xv6-public
SeaBIOS (version 1.16.3-debian-1.16.3-2)


iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1EFC7F60+1EF07F60 CA00



Booting from Hard Disk..xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap sta8
init: starting sh
$ lotterytest
PID     Tickets Ticks
4       30      279
5       20      164
6       10      82
```

Run 2:

```
$ lotterytest


PID     Tickets Ticks
4       30      11394
5       20      7703
6       10      3916
8       30      137
9       20      84
10      10      42
$ $ $
```

Run 3:

```
May 9  20:35
moubarak@ubuntu-os: ~/xv6-public — make qemu-nox
~/xv6-public
SeaBIOS (version 1.16.3-debian-1.16.3-2)


iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1EFC7F60+1EF07F60 CA00



Booting from Hard Disk...
xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
$ lotterytest

PID     Tickets Ticks
4       30      282
5       20      164
6       10      87
$ $
```

Run 4:

```
moubarak@ubuntu-os: ~        moubarak@ubuntu-os: ~/xv6-public — make qemu-nox
SeaBIOS (version 1.16.3-debian-1.16.3-2)


iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1EFC7F60+1EF07F60 CA00



Booting from Hard Disk..xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
$ lotterytest


PID     Tickets Ticks
4       30      324
5       20      126
6       10      86
$ $ $
```

Run 5:

4. Results

The lottery scheduler was tested using five runs with ticket allocations of 30, 20, and 10.

The average CPU tick counts were:

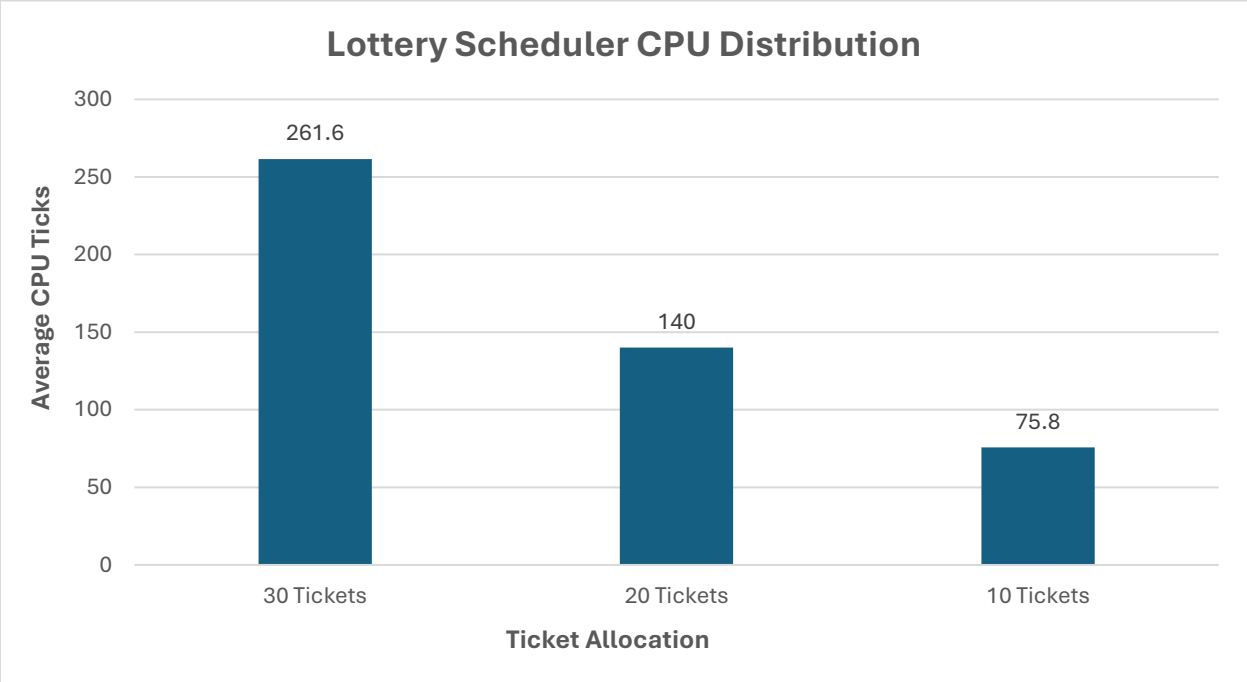
- **30 Tickets:** 261.6 ticks
- **20 Tickets:** 140 ticks
- **10 Tickets:** 75.8 ticks

See the table and Graphs below for more details

Run	30 Tickets	20 Tickets	10 Tickets
1	279	164	82
2	137	84	42
3	282	164	87
4	324	126	86
5	286	162	82
Average	261.6	140	75.8

These results demonstrate that processes with more tickets consistently received more CPU time. The 30-ticket process had the highest execution frequency, followed by the 20-ticket process, while the 10-ticket process had the lowest.

Although individual runs showed slight variations due to random selection, the overall distribution closely matched the expected proportional ratio of 3:2:1 (see figures/graphs below).



- **Figure 1:** Average *CPU ticks for processes with ticket allocations of 30, 20, and 10 over five test runs.*

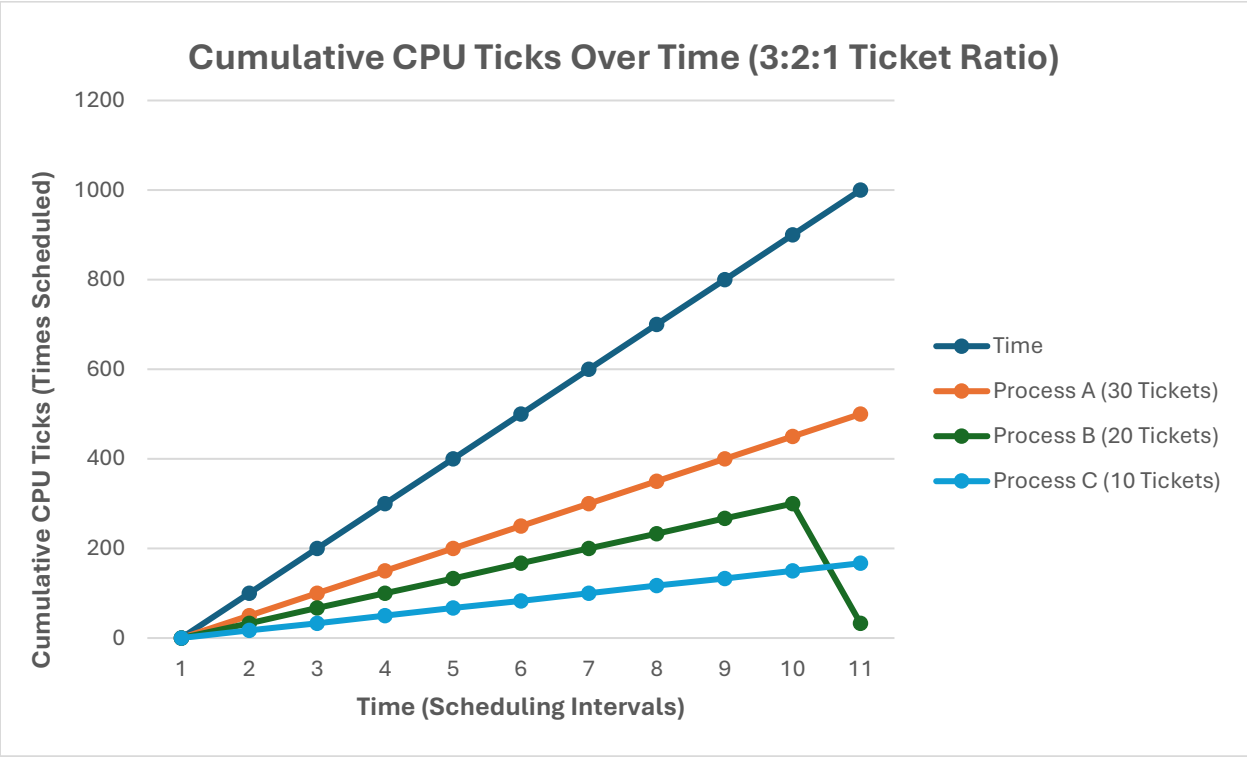


Figure 2: *shows the expected cumulative CPU scheduling frequency over time for processes with ticket allocations of 30, 20, and 10.*

This confirms that the lottery scheduler successfully provides probabilistic fairness and distributes CPU resources according to ticket ownership.

5. Conclusion

This project successfully implemented a lottery scheduler in xv6 by replacing the default round-robin scheduling policy with a ticket-based probabilistic system.

By adding process ticket allocation, scheduling statistics, and the settickets() and getpinfo() system calls, xv6 was transformed into a scheduler capable of allocating CPU time proportionally.

Experimental results confirmed that processes with more tickets consistently received more CPU time, validating the effectiveness of the implementation.

Overall, lottery scheduling proved to be a fair, flexible, and effective alternative to traditional round-robin scheduling, demonstrating how operating systems can dynamically control resource allocation based on process priority.